

AI ASSISTED CODING

SUMANTH POLAM

2303A51121

BACTH – 03

17 – 02 – 2026

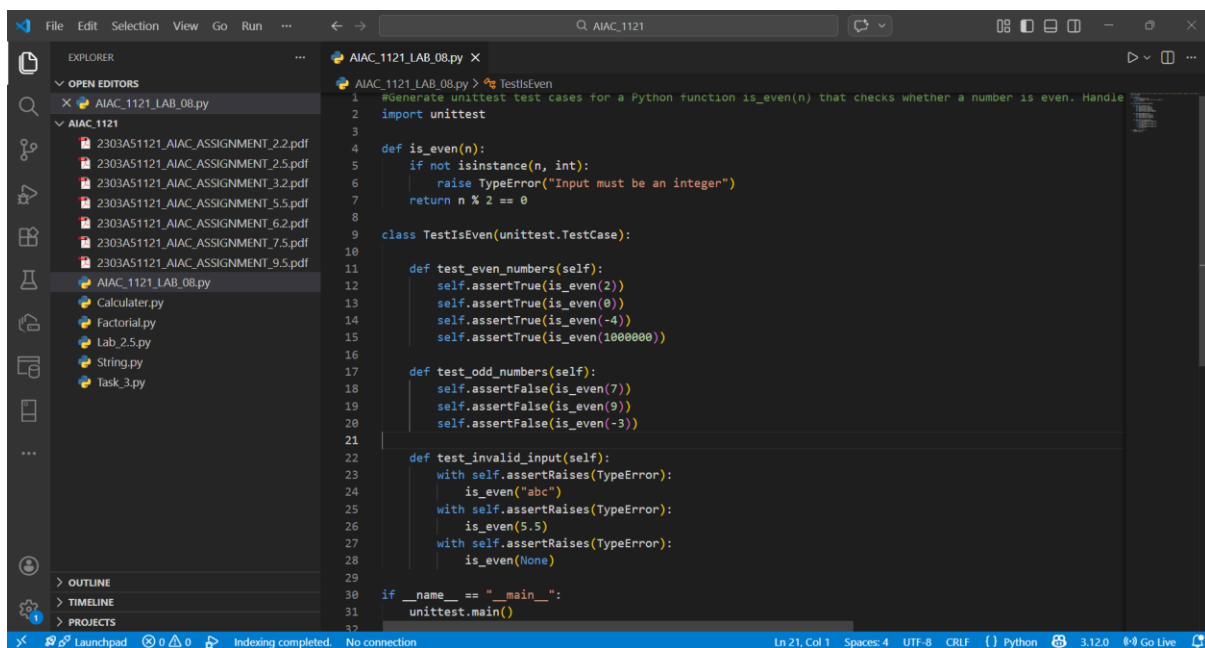
ASSIGNMENT – 08

LAB – 08 : Test – Driven Development with AI – Generating and Working with Test Cases.

Task – 01 : Test – Driven Development for Odd/Even Number Validator.

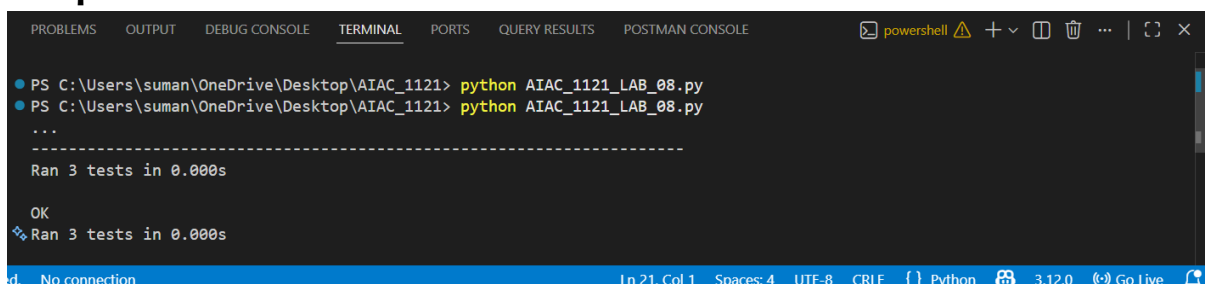
Prompt : Generate unittest test cases for a Python function `is_even(n)` that checks whether a number is even. Handle zero, negative numbers, large integers, and invalid input.

Code :



```
1 #Generate unittest test cases for a Python function is_even(n) that checks whether a number is even. Handle
2 import unittest
3
4 def is_even(n):
5     if not isinstance(n, int):
6         raise TypeError("Input must be an integer")
7     return n % 2 == 0
8
9 class TestIsEven(unittest.TestCase):
10
11     def test_even_numbers(self):
12         self.assertTrue(is_even(2))
13         self.assertTrue(is_even(8))
14         self.assertTrue(is_even(-4))
15         self.assertTrue(is_even(1000000))
16
17     def test_odd_numbers(self):
18         self.assertFalse(is_even(7))
19         self.assertFalse(is_even(9))
20         self.assertFalse(is_even(-3))
21
22     def test_invalid_input(self):
23         with self.assertRaises(TypeError):
24             is_even("abc")
25         with self.assertRaises(TypeError):
26             is_even(5.5)
27         with self.assertRaises(TypeError):
28             is_even(None)
29
30 if __name__ == "__main__":
31     unittest.main()
```

Output :



```
PS C:\Users\suman\OneDrive\Desktop\AIAC_1121> python AIAC_1121_LAB_08.py
PS C:\Users\suman\OneDrive\Desktop\AIAC_1121> python AIAC_1121_LAB_08.py
...
-----
Ran 3 tests in 0.000s

OK
Ran 3 tests in 0.000s
```

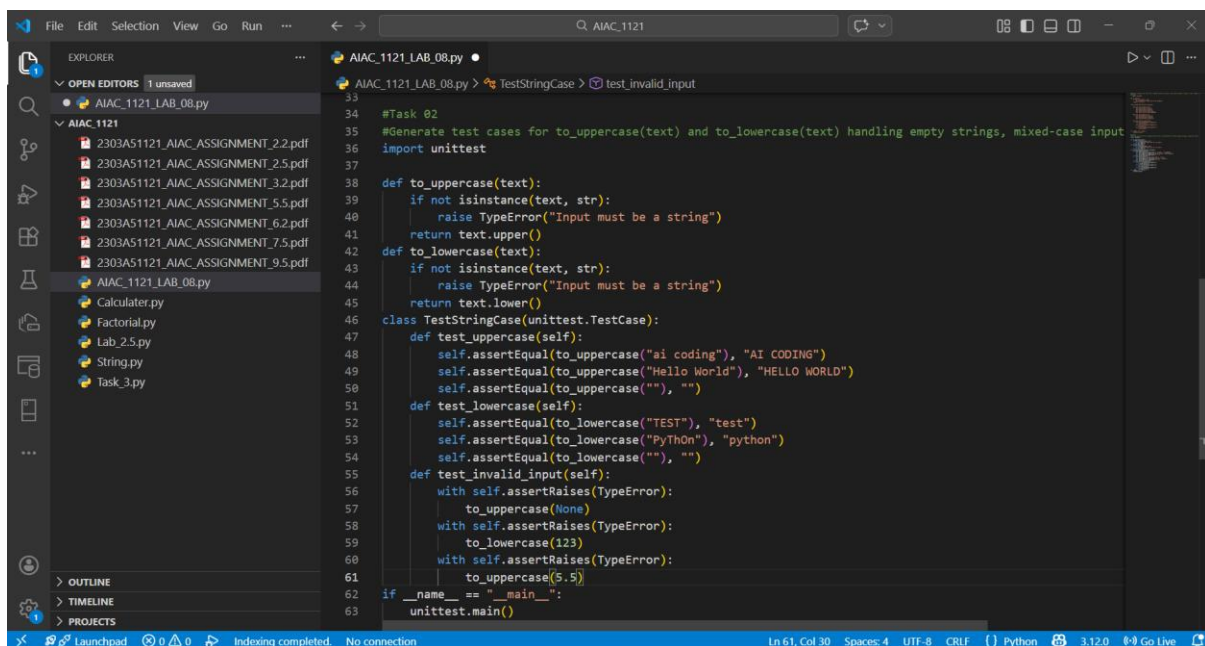
Explanation :

The function first validates that the input is an integer. It then checks divisibility by 2 using modulus operator. It handles zero, negative, and large integers correctly.

Task – 02 : Test – Driven Development for String Case Converter.

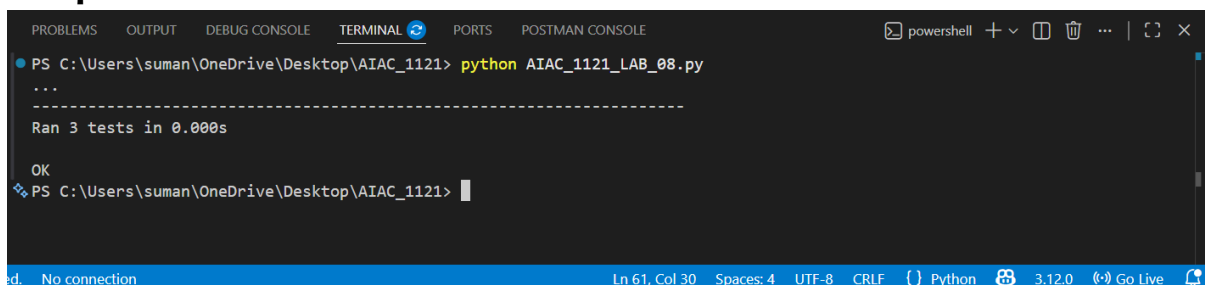
Prompt : Generate test cases for `to_uppercase(text)` and `to_lowercase(text)` handling empty strings, mixed-case input, and invalid inputs.

Code :



```
33
34 #Task 02
35 #Generate test cases for to_uppercase(text) and to_lowercase(text) handling empty strings, mixed-case input
36 import unittest
37
38 def to_uppercase(text):
39     if not isinstance(text, str):
40         raise TypeError("Input must be a string")
41     return text.upper()
42
43 def to_lowercase(text):
44     if not isinstance(text, str):
45         raise TypeError("Input must be a string")
46     return text.lower()
47
48 class TestStringCase(unittest.TestCase):
49     def test_uppercase(self):
50         self.assertEqual(to_uppercase("ai coding"), "AI CODING")
51         self.assertEqual(to_uppercase("Hello World"), "HELLO WORLD")
52         self.assertEqual(to_uppercase(""), "")
53
54     def test_lowercase(self):
55         self.assertEqual(to_lowercase("TEST"), "test")
56         self.assertEqual(to_lowercase("PyThOn"), "python")
57         self.assertEqual(to_lowercase(""), "")
58
59     def test_invalid_input(self):
60         with self.assertRaises(TypeError):
61             to_uppercase(None)
62         with self.assertRaises(TypeError):
63             to_lowercase(123)
64         with self.assertRaises(TypeError):
65             to_uppercase(5.5)
66
67 if __name__ == "__main__":
68     unittest.main()
```

Output:



```
PS C:\Users\suman\OneDrive\Desktop\AIAC_1121> python AIAC_1121_LAB_08.py
...
-----
Ran 3 tests in 0.000s

OK
PS C:\Users\suman\OneDrive\Desktop\AIAC_1121>
```

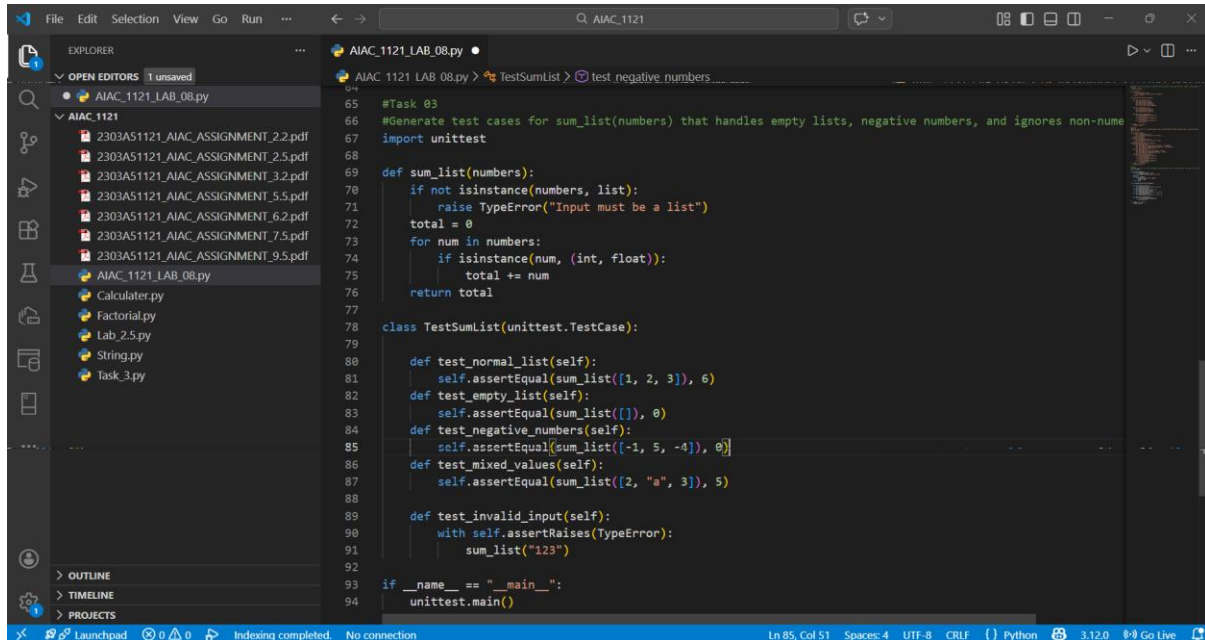
Explanation :

Both functions validate input type and use built-in string methods `.upper()` and `.lower()` for conversion.

Task – 03 : Test – Driven Development for List sum Calculator.

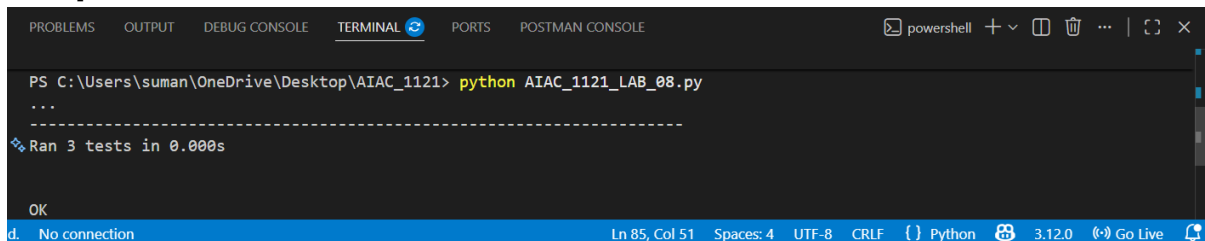
Prompt : Generate test cases for `sum_list(numbers)` that handles empty lists, negative numbers, and ignores non-numeric values.

Code :



```
65 #Task 03
66 #Generate test cases for sum_list(numbers) that handles empty lists, negative numbers, and ignores non-numeric values.
67 import unittest
68
69 def sum_list(numbers):
70     if not isinstance(numbers, list):
71         raise TypeError("Input must be a list")
72     total = 0
73     for num in numbers:
74         if isinstance(num, (int, float)):
75             total += num
76     return total
77
78 class TestSumList(unittest.TestCase):
79
80     def test_normal_list(self):
81         self.assertEqual(sum_list([1, 2, 3]), 6)
82     def test_empty_list(self):
83         self.assertEqual(sum_list([]), 0)
84     def test_negative_numbers(self):
85         self.assertEqual(sum_list([-1, 5, -4]), 0)
86     def test_mixed_values(self):
87         self.assertEqual(sum_list([2, "a", 3]), 5)
88
89     def test_invalid_input(self):
90         with self.assertRaises(TypeError):
91             sum_list("123")
92
93 if __name__ == "__main__":
94     unittest.main()
```

Output :



```
PS C:\Users\suman\OneDrive\Desktop\AIAC_1121> python AIAC_1121_LAB_08.py
...
-----
Ran 3 tests in 0.000s

OK
```

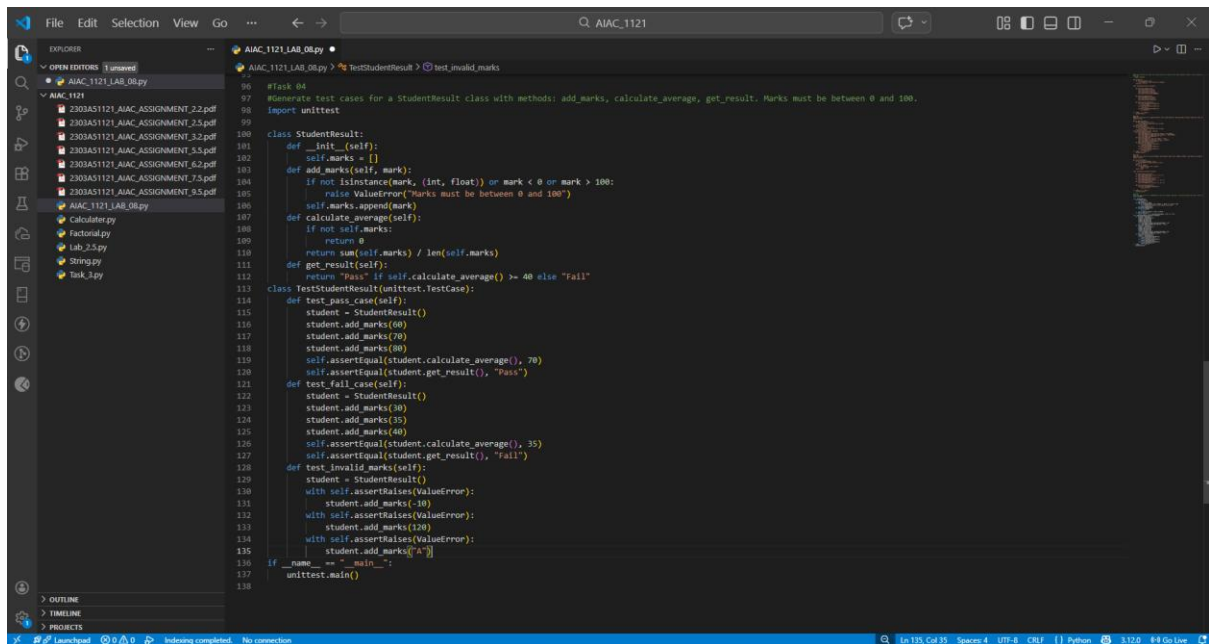
Explanation :

The function iterates through the list and adds only numeric values. It safely ignores non-numeric elements and returns 0 for empty lists.

Task – 04 : Test Cases for Student Result Class.

Prompt : Generate test cases for a `StudentResult` class with methods: `add_marks`, `calculate_average`, `get_result`. Marks must be between 0 and 100.

Code :

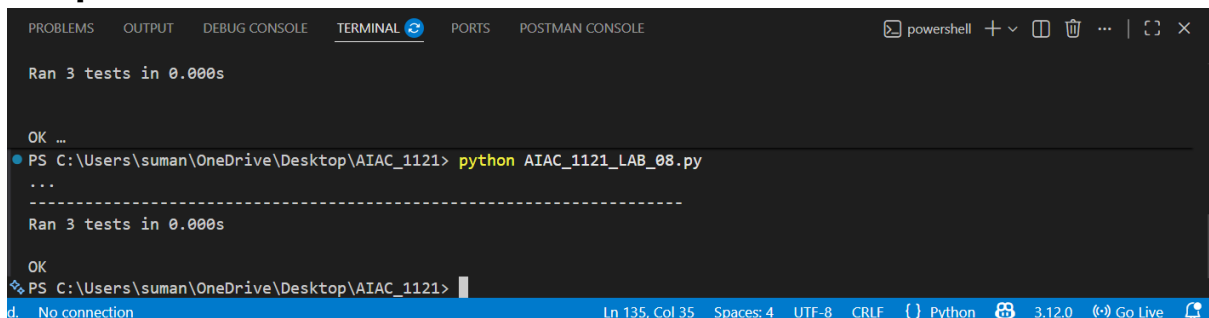


```
#Task 04
#Generate test cases for a StudentResult class with methods: add_marks, calculate_average, get_result. Marks must be between 0 and 100.
import unittest

class StudentResult:
    def __init__(self):
        self.marks = []
    def add_marks(self, mark):
        if not isinstance(mark, (int, float)) or mark < 0 or mark > 100:
            raise ValueError("Marks must be between 0 and 100")
        self.marks.append(mark)
    def calculate_average(self):
        if not self.marks:
            return 0
        return sum(self.marks) / len(self.marks)
    def get_result(self):
        return "Pass" if self.calculate_average() >= 40 else "Fail"

class TestStudentResult(unittest.TestCase):
    def test_pass_case(self):
        student = StudentResult()
        student.add_marks(60)
        student.add_marks(70)
        student.add_marks(80)
        self.assertEqual(student.calculate_average(), 70)
        self.assertEqual(student.get_result(), "Pass")
    def test_fail_case(self):
        student = StudentResult()
        student.add_marks(30)
        student.add_marks(35)
        student.add_marks(40)
        self.assertEqual(student.calculate_average(), 35)
        self.assertEqual(student.get_result(), "Fail")
    def test_invalid_marks(self):
        student = StudentResult()
        with self.assertRaises(ValueError):
            student.add_marks(-10)
        with self.assertRaises(ValueError):
            student.add_marks(100)
        with self.assertRaises(ValueError):
            student.add_marks("A")
    if __name__ == "__main__":
        unittest.main()
```

Output:



```
Ran 3 tests in 0.000s

OK ...
PS C:\Users\suman\OneDrive\Desktop\AIAC_1121> python AIAC_1121_LAB_08.py
...
Ran 3 tests in 0.000s

OK
PS C:\Users\suman\OneDrive\Desktop\AIAC_1121>
```

Explanation :

The class validates marks before storing them. It calculates average dynamically and determines result based on 40% threshold.

Task – 05 : Test – Driven Development for username Validator.

Prompt : Generate test cases for `is_valid_username(username)` with minimum 5 characters, no spaces, and only alphanumeric characters.

The screenshot displays the PyCharm IDE environment. The main editor window shows the file `AIAC_1121_LAB_08.py` with the following Python code:

```

139 #Task 05
140 #Generate test cases for is_valid_username(username) with minimum 5 characters, no spaces, and only alphanumeric characters
141 import unittest
142
143 def is_valid_username(username):
144     if not isinstance(username, str):
145         return False
146     if len(username) < 5:
147         return False
148     if " " in username:
149         return False
150     if not username.isalnum():
151         return False
152     return True
153
154 class TestUsernameValidator(unittest.TestCase):
155     def test_valid_username(self):
156         self.assertTrue(is_valid_username("user01"))
157         self.assertTrue(is_valid_username("abcde"))
158     def test_short_username(self):
159         self.assertFalse(is_valid_username("a1"))
160         self.assertFalse(is_valid_username("user"))
161     def test_space_in_username(self):
162         self.assertFalse(is_valid_username("user name"))
163     def test_special_characters(self):
164         self.assertFalse(is_valid_username("user@123"))
165         self.assertFalse(is_valid_username("user#1"))
166     def test_invalid_type(self):
167         self.assertFalse(is_valid_username(None))
168         self.assertFalse(is_valid_username(12345))
169
170 if __name__ == "__main__":
171     unittest.main()

```

The left sidebar shows the project structure with the following files:

- AIAC_1121_LAB_08.py
- AIAC_1121
 - 2303AS1121_AIAC_ASSIGNMENT_2.2.pdf
 - 2303AS1121_AIAC_ASSIGNMENT_2.5.pdf
 - 2303AS1121_AIAC_ASSIGNMENT_3.2.pdf
 - 2303AS1121_AIAC_ASSIGNMENT_5.5.pdf
 - 2303AS1121_AIAC_ASSIGNMENT_6.2.pdf
 - 2303AS1121_AIAC_ASSIGNMENT_7.5.pdf
 - 2303AS1121_AIAC_ASSIGNMENT_9.5.pdf
- Calculator.py
- Factorial.py
- Lab_2.5.py
- String.py
- Task_3.py

The bottom status bar indicates the current position is `Ln 167, Col 51` and the file is encoded in `UTF-8` with `CRLF` line endings. The status bar also shows `Python` and `3.12.0`.

The screenshot shows a VS Code terminal window with the following content:

```
PS C:\Users\suman\OneDrive\Desktop\AIAC_1121> python AIAC_1121_LAB_08.py
.....
-----
Ran 5 tests in 0.000s

OK
PS C:\Users\suman\OneDrive\Desktop\AIAC_1121>
```

The terminal window has tabs for PROBLEMS, OUTPUT, DEBUG CONSOLE, TERMINAL (active), PORTS, and POSTMAN CONSOLE. The status bar at the bottom shows "d. No connection", "Ln 169, Col 20", "Spaces: 4", "UTF-8", "CRLF", "{ } Python", "3.12.0", and "Go Live".

The function checks length, space restriction, and alphanumeric condition using built-in string validation methods.

THANK YOU!!